Kerim Turak

Ankara Yıldırım Beyazıt University
EEE Department - Spring, 20-21
EE226 Introduction to Microprocessors Laboratory
Universal Asynchronous Receiver-Transmitter (UART) and Analog to Digital Converter (ADC)

Implementation Video: https://youtu.be/PROccPgmiGg
Code link :
https://drive.google.com/file/d/19NjN7wpOTySLUg4tCRH3l8517Nl6O3a6/view?usp=sharing

1. Introduction

In this lab, we used the LM 35 temperature sensor and USB to TLL device to understand the UART communication with working principle of ADC and USB to TLL device communication logic (by using RX, TX connection)

LM 35

The LM35 series are precision integrated-circuit temperature sensors,
whose output voltage is linearly proportional to the Celsius or Centigrade temperature.

Temp. = 3300/40950 (Vcc/4095/10)

USB TO TLL

The **PL-2303 chip** provides a convenient solution for connecting a UART-like full-duplex asynchronous serial device to any Universal Serial Bus (USB) capable host. The PL-2303 highly compatible drivers could successfully simulate generic COM ports on most operating systems.

Setting up the Algorithm and Writing the Code

Firstly, we need to know which module or part of the launchpad will be used in this task. We will mainly use UART, TIMER, ADC logic and we have to initialize necessary setting for this task. I create a template as following.

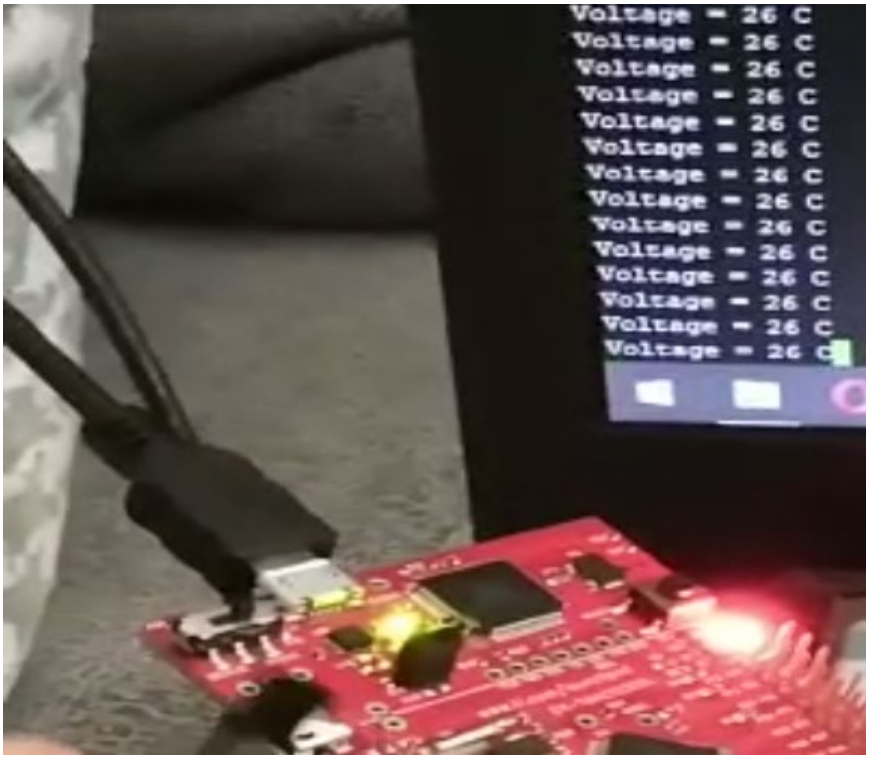
```c
/*header files*/
#include <stdint.h>
#include "tm4c123gh6pm.h"
#include "TM4C123GH6PM.h"
#include <stdio.h>
void Delay(unsigned long counter);
void UART5_init(void);
void UART5_Transmitter(unsigned char data);
void printstring(char *str);
void PortFInit(void);
char mesg[12];
volatile unsigned int voltage;
volatile unsigned int sensor ;
volatile unsigned int adc_value;
```

I set the TIMER, UART and ADC by defining functions.

Main part of the my code is as following that part calculation and printing are done with the help of the UART and ADC.

```c
    while(1){
        /* convert digital value back into voltage */
       voltage = adc_value*3300/40950;
       sprintf(mesg, "\r\nVoltage = %d C", voltage);
       printstring(mesg);
       Delay(600);
      if(voltage < 27 ){
        GPIO_PORTF_DATA_R = 0x02;
      }
      else if (voltage <= 29){
        GPIO_PORTF_DATA_R = 0x08;
      }
      else{
        GPIO_PORTF_DATA_R = 0x0A;
      }
    }
```

Output:



2. Result and Discussion

To complete this lab, I first learned the USB TO PLL logic. I learned how to make two important connections such as RX, TX and communication between 2 UART units. With the LM 35, I better understood how the temperature is calculated. I have understood the relationship between the sensor's receiver and transmitter with logic of ADC.

3. Conclusion

I saw and figured out the ADC and UART together. We evaluate the temperature than transfer this information with the UART. Temperature sensors have a working range and resolution can be determined with these working ranges.

4. References

a. https://www.electronics-tutorials.ws/io/io_3.html
b. https://tr.farnell.com/texas-instruments/boostxl-senshub/mpu9150-sensor-hub-boosterpack/dp/2314938
c. https://microcontrollerslab.com/hc-sr04-ultrasonic-sensor-interfacing-with-tm4c123-tiva-c-launchpad/
d.

Appendix

```
/*header files*/
#include <stdint.h>
#include "tm4c123gh6pm.h"
#include "TM4C123GH6PM.h"
#include <stdio.h>
void Delay(unsigned long counter);
void UART5_init(void);
void UART5_Transmitter(unsigned char data);
void printstring(char *str);
void PortFInit(void);
char mesg[12];
volatile unsigned int voltage;
volatile unsigned int sensor ;
volatile unsigned int adc_value;
void ADC0SS3_Handler(void){
        adc_value = ADC0_SSFIFO3_R;                /* read adc coversion result from SS3
FIFO*/
        ADC0_ISC_R = 8;                                    /* clear coversion clear flag bit*/
  ADC0_PSSI_R  |= (1<<3);        /* Enable SS3 conversion or start sampling data from AN0
*/
}

int main(void){
            UART5_init();
                    PortFInit();
   /* Enable Clock to ADC0 and GPIO pins*/
   SYSCTL_RCGCGPIO_R |= (1<<4);   /* Enable Clock to GPIOE or PE3/AN0 */
                  SYSCTL_RCGCADC_R |= (1<<0);    /* AD0 clock enable*/
   /* initialize PE3 for \AIN0 input  */
   GPIO_PORTE_AFSEL_R |= (1<<3);      /* enable alternate function */
   GPIO_PORTE_DEN_R &= ~(1<<3);       /* disable digital function */
   GPIO_PORTE_AMSEL_R |= (1<<3);      /* enable analog function */
   /* initialize sample sequencer3 */
   ADC0_ACTSS_R&= ~(1<<3);       /* disable SS3 during configuration */
   ADC0_EMUX_R &= ~0xF000;                    /* software trigger conversion */
   ADC0_SSMUX3_R = 0;                        /* get input from channel 0 */
   ADC0_SSCTL3_R |= (1<<1)|(1<<2);       /* take one sample at a time, set flag at 1st
sample */
                  /* Enable ADC Interrupt */
                  ADC0_IM_R |= (1<<3); /* Unmask ADC0 sequence 3 interrupt*/
                  NVIC_EN0_R = 1 << 17; /* enable IRQ17 for ADC0SS3*/
                  ADC0_ACTSS_R |= (1<<3);        /* enable ADC0 sequencer 3 */
                  ADC0_PSSI_R |= (1<<3);       /* Enable SS3 conversion or start sampling data
from AN0 */
   while(1){
                           /* convert digital value back into voltage */
                          voltage = adc_value*3300/40950;
                          sprintf(mesg, "\r\nVoltage = %d C", voltage);
```

```
                        printstring(mesg);
    Delay(600);
                        if(voltage < 27 ){
                                GPIO_PORTF_DATA_R = 0x02;
                        }
                        else if (voltage <= 29){
                                GPIO_PORTF_DATA_R = 0x08;
                        }
                        else{
                                GPIO_PORTF_DATA_R = 0x0A;
                        }
  }
}

void UART5_init(void){
                SYSCTL_RCGCUART_R |= 0x20;  /* enable clock to UART5 */
  SYSCTL_RCGCGPIO_R |= 0x10;  /* enable clock to PORTE for PE4/Rx and RE5/Tx */
  Delay(1);
  /* UART0 initialization */
  UART5_CTL_R = 0;        /* UART5 module disbable */
  UART5_IBRD_R = 104;     /* for 9600 baud rate, integer = 104 */
  UART5_FBRD_R = 11;      /* for 9600 baud rate, fractional = 11*/
  UART5_CC_R = 0;         /*select system clock*/
  UART5_LCRH_R = 0x60;    /* data lenght 8-bit, not parity bit, no FIFO */
  UART5_CTL_R = 0x301;    /* Enable UART5 module, Rx and Tx */
  /* UART5 TX5 and RX5 use PE4 and PE5. Configure them digital and enable alternate
function */
  GPIO_PORTE_DEN_R = 0x30;     /* set PE4 and PE5 as digital */
  GPIO_PORTE_AFSEL_R = 0x30;   /* Use PE4,PE5 alternate function */
  GPIO_PORTE_AMSEL_R = 0;   /* Turn off analg function*/
  GPIO_PORTE_PCTL_R = 0x00110000;    /* configure PE4 and PE5 for UART */
}
void UART5_Transmitter(unsigned char data){
  while((UART5_FR_R & (1<<5)) != 0);  /* wait until Tx buffer not full */
  UART5_DR_R = data;                  /* before giving it another byte */
}

void printstring(char *str){
 while(*str){
                UART5_Transmitter(*(str++));
        }
}

void Delay(unsigned long counter){
        unsigned long i = 0;
        for(i=0; i< counter*1000; i++);
}

void PortFInit(void) {
  SYSCTL_RCGC2_R   |= 0x00000020;    // 1) F clock
```

```
  GPIO_PORTF_LOCK_R  = 0x4C4F434B;     // 2) unlock Port F
  GPIO_PORTF_CR_R   |= 0x1F;           // allow changes to PF4-PF0
  GPIO_PORTF_AMSEL_R = 0x00;           // 3) disable analog function
  GPIO_PORTF_PCTL_R  = 0x00;           // 4) GPIO clear bit PCTL
  GPIO_PORTF_DIR_R  |= 0x0E;           // 5) PF4, PF0 inputs. PF3, PF2, PF1 outputs
  GPIO_PORTF_AFSEL_R = 0x00;           // 6) no alternate function
  GPIO_PORTF_PUR_R  |= 0x11;           // enable pullup resistors on PF4, PF0
  GPIO_PORTF_DEN_R  |= 0x1F;           // 7) enable digital pins PF4-PF0
}
```